

بسمه تعالی

گزارش کار پروژه اختیاری سیستم عامل

Process Profiling in eBPF

استاد
دکتر سلطانی

نویسندگان
جواد باجلان

دانشگاه شهید مدنی آذربایجان
بهار 1404

فهرست

صفحه	عنوان
۳	مقدمه.....
۳	اهمیت مسئله در سیستم عامل.....
۳	فیلتر بسته های گسترده بر کلی.....
۴	فراخوانی های سیستم و اهمیت آن ها.....
۵	مدیریت حافظه لینوکس.....
۶	اجزا پروژه.....
۷	جزئیات پیاده سازی.....
۱۱	تحلیل کد.....
۲۵	ساخت برنامه.....
۲۵	درک خروجی.....
۲۷	تصاویر.....

پروفایلر فرآیند ابزاری برای نظارت و تحلیل رفتار زمان اجرای فرآیندها در سیستم عامل لینوکس طراحی شده است. این ابزار از تکنولوژی قدرتمند فیلتر بسته‌های برکلی (eBPF) استفاده می‌کند تا بینش عمیقی از اجرای فرآیند، استفاده از فراخوانیهای سیستم، الگوهای تخصیص حافظه و نشت‌های بالقوه حافظه ارائه دهد.

اهمیت مسئله در سیستم عامل

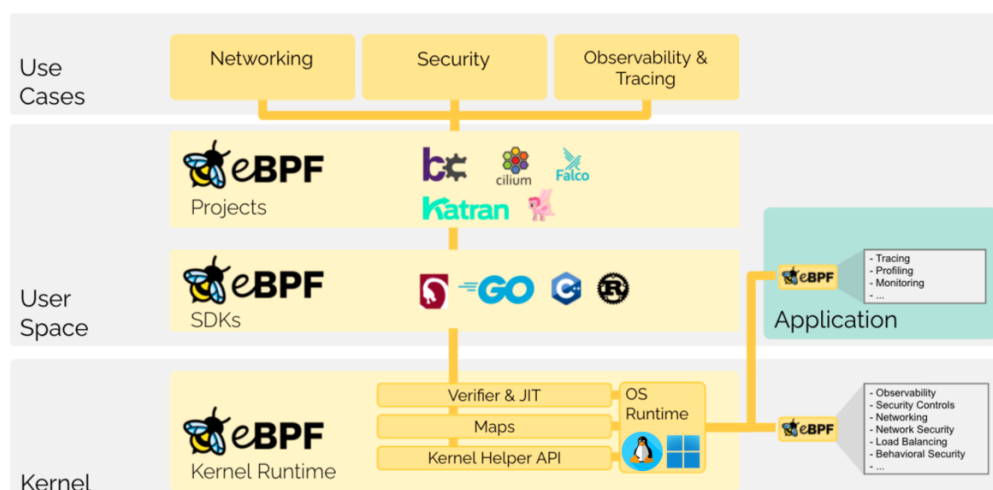
این مسئله به وضوح مسائل زیر را نشان می‌دهد:

- مدیریت فرآیند: چگونگی ایجاد، اجرا و پایان فرآیندها
- فراخوانی‌های سیستم: رابط بین برنامه‌های کاربردی و سیستم عامل
- مدیریت حافظه: تخصیص، آزادسازی و نشت بالقوه حافظه
- ردیابی منابع: نظارت و تحلیل الگوهای استفاده از منابع
- تعامل فضای هسته-کاربر: چگونگی تعامل برنامه‌ها با هسته

فیلتر بسته‌های گسترده برکلی (eBPF)

تکنولوژی انقلابی است که اجازه اجرای برنامه‌ها در هسته لینوکس را بدون تغییر در کد منبع هسته یا بارگذاری ماژول‌های هسته می‌دهد. این تکنولوژی جانشینی برای مکانیزم قبلی فیلترینگ در لینوکس به نام BPF می‌باشد.

ایمنی اجرا کد توسط یک مکانیزم تایید کننده در داخل هسته لینوکس که تجزیه کد ایستا را انجام می‌دهد، تضمین می‌شود، به این صورت که برنامه‌هایی که تداخل دارند یا هنگ می‌کنند را رد می‌کند. این مدل اعتبارسنجی با محیط‌های sandbox متفاوت است. نمونه برنامه‌هایی که در این اعتبارسنجی رد می‌شوند برنامه‌هایی هستند که توقف آن‌ها تضمین شده نیست، مانند حلقه `while` بدون شرط خروج که باعث گیر افتادن برنامه در حلقه می‌شود.



eBPF از طریق مجموعه‌از اجزا کار می‌کند:

- ماشین مجازی **eBPF**: زمان اجرای درون‌هسته‌ای که بایت‌کد eBPF را اجرا می‌کند.
- تایید کننده: اطمینان از ایمنی اجرای برنامه‌ها.
- کامپایلر **JIT**: ترجمه بایت‌کد eBPF به کد ماشین بومی برای اجرای کارآمد.
- **Map**ها: ذخیره‌سازهای key-value که اشتراک داده بین برنامه‌های eBPF و فضای کاربر را امکان پذیر می‌کنند.
- **Helper**ها: توابع هسته که برنامه‌های eBPF می‌توانند فراخوانی کنند.
- **Hook**ها: نقاط مختلف در هسته که برنامه‌های eBPF می‌توانند متصل شوند.

پروفایلر فرآیند از eBPF برای اتصال به فراخوانی‌های سیستم، توابع مدیریت حافظه و رویدادهای چرخه فرآیند استفاده می‌کند

دو جز فضای کاربر و فضای هسته از طریق مکانیزم‌های زیر با هم در ارتباط هستند:

- **بافر حلقه‌ای**: برای انتقال داده‌های رویدادها از فضای هسته به فضای کاربر
- **Hash map**: برای ذخیره داده‌های پیکربندی (مانند PIDها هدف) و اطلاعات وضعیت
- **توابع کتابخانه BPF**: برای بارگذاری، تایید و اتصال برنامه‌های eBPF

فراخوانی‌های سیستم و اهمیت آنها

فراخوانی‌های سیستم رابط بنیادی بین برنامه‌های کاربر و هسته سیستم‌عامل هستند. آنها نقطه ورود کنترل شده‌ای برای برنامه‌ها فراهم می‌کنند تا خدمات هسته مانند عملیات فایل، کنترل فرآیند، ارتباط شبکه و مدیریت حافظه را درخواست کنند

مکانیزم فراخوانی سیستم به دلایل متعددی حیاتی است:

- **حفاظت**: فراخوانی‌های سیستم جداسازی بین فضای کاربر و فضای هسته را اجرا می‌کنند و امنیت و پایداری سیستم را حفظ می‌کنند.
- **انتزاع**: رابط یکنواختی برای خدمات هسته فراهم می‌کنند و پیچیدگی تعاملات سخت‌افزاری را پنهان می‌کنند
- **مدیریت منابع**: فراخوانی‌های سیستم به هسته اجازه کنترل و تخصیص منابع بین فرآیندهای رقیب را می‌دهند.
- **قابلیت حمل**: برنامه‌هایی که برای استفاده از فراخوانی‌های سیستم استاندارد نوشته شده‌اند می‌توانند بر روی پیاده‌سازی‌های مختلف همان سیستم‌عامل اجرا شوند (POSIX)

جریان معمولی یک فراخوانی سیستم شامل:

۱. برنامه آرگومان‌هایی برای فراخوانی سیستم آماده می‌کند
۲. دستورالعمل خاصی (مثل `syscall` در `x86_64`) برای انتقال به حالت هسته اجرا می‌کند
۳. هسته آرگومان‌ها را تایید می‌کند و عملیات درخواست شده را انجام می‌دهد

۴. هسته نتیجه را به فضای کاربر برمی گرداند و به حالت کاربر برمی گردد

نظارت بر فراخوانی‌های سیستم بینش‌های ارزشمندی از رفتار برنامه، الگوهای استفاده از منابع و گلوگاه‌های بالقوه فراهم می‌کند. پروفایلر فرآیند هم ورود و هم خروج فراخوانی‌های سیستم را ردیابی می‌کند و نوع فراخوانی، آرگومان‌ها، مقادیر بازگشتی و اطلاعات زمان‌بندی را ثبت می‌کند.

مدیریت حافظه در لینوکس

مدیریت حافظه یکی از پیچیده‌ترین و حیاتی‌ترین توابع سیستم عامل است. لینوکس از سیستم حافظه مجازی پیچیده‌ای استفاده می‌کند که هر فرآیند را با فضای آدرس مجازی خود فراهم می‌کند، فرآیندها را از یکدیگر محافظت می‌کند و استفاده کارآمد از حافظه فیزیکی را امکان‌پذیر می‌کند.

جنبه‌های کلیدی مدیریت حافظه لینوکس شامل:

- **حافظه مجازی:** هر فرآیند فضای آدرس مجازی خود را دارد که توسط واحد مدیریت حافظه (MMU) به حافظه فیزیکی نگاشت می‌شود
- **صفحه‌بندی:** حافظه به صفحات با اندازه ثابت (معمولاً 4KB) تقسیم می‌شود که می‌توانند به صورت جداگانه نگاشت، عدم نگاشت و جایگزین شوند.
- **خطاهای صفحه:** وقتی فرآیند به آدرس مجازی دسترسی پیدا می‌کند که در حال حاضر به حافظه فیزیکی نگاشت نشده است، خطای صفحه رخ می‌دهد و هسته را وادار می‌کند صفحه مورد نیاز را بارگذاری کند.
- **تخصیص حافظه:** فرآیندها حافظه را از طریق فراخوانی‌های سیستم مانند `mmap`، `brk` و توابع کتابخانه‌ای مانند `malloc` درخواست می‌کنند.
- **بازیابی حافظه:** هسته وقتی حافظه سیستم کم است، حافظه را از فرآیندها بازیابی می‌کند و صفحات کم‌تر مورد استفاده شده را به دیسک جایگزین می‌کند.
- **حافظه مشترک:** چندین فرآیند می‌توانند صفحات فیزیکی یکسان را نگاشت کنند و ارتباط کارآمد بین فرآیندی را امکان‌پذیر کنند.

مسائل رایج در مدیریت حافظه شامل:

- **نشت حافظه:** وقتی فرآیند حافظه تخصیص می‌دهد اما هرگز آن را آزاد نمی‌کند
- **تکه‌تکه شدن:** وقتی حافظه آزاد به قطعات کوچک و غیرپیوسته تقسیم می‌شود
- **درگیری:** فعالیت صفحه‌بندی مفرط به دلیل حافظه فیزیکی ناکافی
- **تخصیص بیش از حد:** وقتی فرآیند حافظه بیشتری از آن چه فیزیکی در دسترس است، درخواست می‌کند.

اجزا پروژه

- فایل‌های هدر:
 - **process_profiler.h**
ساختار داده و ثوابت مشترک استفاده شده توسط هر دو جز هسته و فضای کاربر را تعریف می‌کند
 - **vmlinux.h**
هدر تولید شده خودکار حاوی تعاریف ساختار داده هسته
- برنامه **eBPF**:
 - **process_profiler.bpf.c**
شامل برنامه‌های eBPF که به hookهای مختلف هسته متصل می‌شوند و داده جمع‌آوری می‌کنند.
- برنامه فضای کاربر:
 - **Main.c**
جز فضای کاربر را پیاده‌سازی می‌کند که برنامه‌های eBPF را بارگذاری می‌کند، داده‌ها را جمع‌آوری و تحلیل می‌کند و نتایج را ارائه می‌دهد.
- سیستم ساخت:
 - **Makefile**
فرآیند ساخت شامل کامپایل برنامه eBPF و برنامه فضای کاربر را پیگیربندی می‌کند.
- **Map**ها و ساختارهای داده:
 - **بافر حلقه‌ای رویدادها**: برای انتقال داده‌های رویداد از فضای هسته به فضای کاربر
 - **نقشه pidهای هدف**: برای ذخیره pidهای فرآیندهایی که باید نظارت شوند.
 - **نقشه حافظه**: برای ردیابی تخصیص‌های حافظه و تشخیص نشت‌های بالقوه
 - **نقشه زمان ورود فراخوانی سیستم**: برای اندازه‌گیری مدت زمان فراخوانی سیستم

این اجزا با هم کار می‌کنند تا راه‌حل جامع پروفایل‌گیری فرآیند ارائه دهند.

جزئیات پیاده سازی

فایل‌های هدر و ساختارهای داده

پروفاایلر فرآیند از چندین ساختار داده کلیدی تعریف شده در فایل‌های هدر خود استفاده می‌کند:

- شمارش انواع رویداد (**enum event_type**):
 - انواع مختلف رویدادهایی که می‌توانند توسط پروفاایلر ضبط شوند را تعریف می‌کند
 - شامل رویدادهای چرخه حیات فرآیند، رویدادهای فراخوانی سیستم، عملیات حافظه و خطاهای صفحه
 - ساختار داده رویداد (**struct event_data**):
 - حاوی اطلاعات تفصیلی در مورد رویدادهای ضبط شده
 - شامل زمان، شناسه فرآیند، شناسه نخ، نوع رویداد و داده‌های خاص رویداد
 - برای انتقال اطلاعات از فضای هسته به فضای کاربر استفاده می‌شود
 - ساختار تخصیص حافظه (**struct memory_alloc**):
 - اطلاعات مربوط به تخصیص‌های حافظه را ردیابی می‌کند
 - برای تشخیص نشت حافظه استفاده می‌شود
 - آدرس، اندازه، زمان و شناسه فرآیند تخصیص‌ها را ذخیره می‌کند
 - ساختار نگاشت فراخوانی سیستم (**struct syscall_map**):
 - شماره فراخوانی‌های سیستم را به نام‌هایشان نگاشت می‌کند
 - گزارش قابل خواندن توسط انسان از فعالیت فراخوانی سیستم را امکان‌پذیر می‌کند
 - برای معماری‌های مختلف (x86_64 و aarch64) به طور جداگانه تعریف شده
 - توضیحات آرگومان‌های فراخوانی سیستم (**struct syscall_arg_desc**):
 - نام و نوع آرگومان‌های فراخوانی سیستم را توضیح می‌دهد
 - برای قالب‌بندی اطلاعات فراخوانی سیستم به شکل قابل خواندن استفاده می‌شود
- این ساختارهای داده دقیقاً طراحی شده‌اند تا تعادل بین جمع‌آوری داده جامع و کارایی را برقرار کنند و سربار را کمینه کرده و اطلاعات تفصیلی ارائه دهند.

برنامه‌های eBPF

جز eBPF پروفایلر فرآیند شامل چندین برنامه eBPF است که به hookهای مختلف هسته متصل می‌شوند:

- نظارت چرخه حیات فرآیند:
 - **trace_process_exec**: به نقطه ردیابی `sched_process_exec` متصل می‌شود تا ایجاد فرآیند را تشخیص دهد.
 - **trace_process_exec_binprm**: به `bprm_execve` با `kprobe` به عنوان روش جایگزین برای تشخیص ایجاد فرآیند متصل می‌شود.
 - **trace_process_exit**: به نقطه ردیابی `sched_process_exit` متصل می‌شود تا پایان فرآیند را تشخیص دهد.
- نظارت فراخوانی سیستم:
 - **trace_sys_enter**: به نقطه ردیابی `raw_syscalls/sys_enter` متصل می‌شود تا ورود فراخوانی سیستم را ضبط کند.
 - **trace_sys_exit**: به نقطه ردیابی `raw_syscalls/sys_exit` متصل می‌شود تا خروج فراخوانی سیستم را ضبط کند.
- نظارت عملیات حافظه:
 - **trace_malloc** و **trace_malloc_ret**: به `uprobe` روی تابع `malloc` متصل می‌شوند تا تخصیص‌های حافظه را ردیابی کنند.
 - **trace_free**: به `uprobe` روی تابع `free` متصل می‌شود تا آزادسازی حافظه را ردیابی کند.
 - **trace_mmap_syscall**: به نقطه ردیابی `syscalls/sys_enter_mmap` متصل می‌شود تا عملیات نگاشت حافظه را ردیابی کند.
 - **trace_brk_enter**: به نقطه ردیابی `syscalls/sys_enter_brk` متصل می‌شود تا گسترش `heap` را ردیابی کند.
- رویدادهای مدیریت حافظه:
 - **trace_kmalloc**: به نقطه ردیابی `kmem/kmalloc` متصل می‌شود تا تخصیص‌های حافظه هسته را ردیابی کند.
 - **trace_kfree**: به نقطه ردیابی `kmem/kfree` متصل می‌شود تا آزادسازی حافظه هسته را ردیابی کند.
 - **trace_reclaim_begin**: به نقطه ردیابی `vmscan/mm_vmscan_direct_reclaim_begin` متصل می‌شود تا بازیابی حافظه را تشخیص دهد.

هر برنامه eBPF الگوی مشابهی دنبال می‌کند:

1. بررسی این‌که آیا فرآیند فعلی در حال پروفایل‌گیری است (با جست‌جوی pid آن در نقشه target_pids)
2. اگر هست، داده‌های مرتبط با رویداد را جمع‌آوری می‌کند
3. ساختار رویداد ایجاد می‌کند و آن را با داده‌های جمع‌آوری شده پر می‌کند
4. رویداد را به بافر حلقه‌ای برای بازیابی توسط جز فضای کاربر ارسال می‌کند

برنامه فضای کاربر

جز فضای کاربر پروفایلر فرآیند مسئول موارد زیر است:

- **مقداردهی اولیه و تنظیم:**
 - تجزیه آرگومان‌های خط فرمان
 - بارگذاری و اتصال برنامه‌های eBPF
 - تنظیم کنترل‌کننده‌های سیگنال برای پایان مناسب
 - اضافه کردن pidهای هدف به نقشه نظارت
- **مدیریت فرآیند:**
 - تولید فرآیندهای هدف در صورت تعیین در خط فرمان
 - انتظار برای تکمیل فرآیند
 - جمع‌آوری آمار فرآیند از /proc
- **مدیریت رویداد:**
 - دریافت رویدادها از بافر حلقه‌ای eBPF
 - پردازش و نمایش اطلاعات رویداد
 - به‌روزرسانی آمار بر اساس رویدادهای دریافتی
- **مدیریت فراخوانی سیستم:**
 - نگاشت شماره فراخوانی‌های سیستم به نام‌ها
 - قالب‌بندی آرگومان‌های فراخوانی سیستم برای نمایش
 - ردیابی آمار فراخوانی سیستم
- **ردیابی حافظه:**
 - به‌روزرسانی آمار استفاده از حافظه
 - تشخیص نشت‌های بالقوه حافظه
 - گزارش معیارهای مرتبط به حافظه
- **Probهای فضای کاربر:**
 - اتصال uprobesها به توابع malloc و free
 - یافتن offsetهای صحیح برای این توابع در کتابخانه مشترک libc
 - مدیریت رویدادهای uprobes برای ردیابی تخصیص حافظه
- **گزارش‌دهی:**

- تولید آمار جامع در مورد فرآیند پروفایل شده
- گزارش نشت‌های حافظه
- نمایش آمار صفحه‌بندی
- ارائه بینش‌های عملکرد

برنامه فضای کاربر حول حلقه رویداد اصلی ساختار یافته است که بافر حلقه‌ای را برای رویدادهای برنامه‌های eBPF نظرسنجی می‌کند، آن‌ها را پردازش می‌کند و آمار را به‌روزرسانی می‌کند. از کنترل‌کننده‌های سیگنال برای تشخیص زمانی که کاربر می‌خواهد پروفایل‌گیری را متوقف کند، استفاده می‌شود و گزارش نهایی با اطلاعات جمع‌آوری شده تولید می‌کند.

سیستم ساخت

سیستم ساخت پروفایل فرآیند از طریق **Makefile** پیکربندی می‌شود که موارد زیر را مدیریت می‌کند:

- مدیریت وابستگی:
 - بررسی کتابخانه‌ها و ابزارهای موردنیاز
 - ارائه هدفی برای نصب وابستگی‌ها
 - اطمینان از در دسترس بودن نسخه‌های صحیح اجزا
- کامپایل برنامه eBPF:
 - تولید هدر **vmlinux.h** با استفاده از **bpftool**
 - کامپایل برنامه eBPF به بایت‌کد BPF
 - تولید هدر اسکلت برای جز فضای کاربر
- کامپایل برنامه فضای کاربر:
 - کامپایل برنامه فضای کاربر
 - لینک کردن با کتابخانه‌های مورد نیاز (**libz**, **libelf**, **libbpf**)
 - Include کردن فایل‌های هدر لازم
- مقداردهی اولیه و تنظیم:
 - تجزیه آرگومان‌های خط فرمان
 - بارگذاری و اتصال برنامه‌های eBPF
 - تنظیم کنترل‌کننده‌های سیگنال برای پایان مناسب
 - اضافه کردن pidهای هدف به نقشه نظارت

فرآیند ساخت این مراحل را دنبال می‌کند:

1. تولید **vmlinux.h** از اطلاعات BTF (BPF Type Format) هسته
2. کامپایل برنامه eBPF به بایت‌کد
3. تولید هدر اسکلت برای برنامه eBPF
4. کامپایل برنامه فضای کاربر
5. لینک کردن همه‌چیز به یک فایل اجرایی نهایی

تحلیل کد

هدر پروفایلر فرآیند

فایل هدر `process_profiler.h` ساختارهای داده بنیادی و ثوابت استفاده شده در سراسر پروژه را تعریف می‌کند:

```
#ifndef PROCESS_PROFILER_H
#define PROCESS_PROFILER_H
#include <stdint.h>
#define MAX_COMM_LEN 16
#define MAX_FILENAME_LEN 256

enum event_type {
    EVENT_PROCESS_START,
    EVENT_PROCESS_END,
    EVENT_SYSCALL_ENTER,
    EVENT_SYSCALL_EXIT,
    EVENT_MEMORY_ALLOC,
    EVENT_MEMORY_FREE,
    EVENT_PAGE_FAULT,
    EVENT_MMAP,
    EVENT_MUNMAP
};

struct event_data {
    uint64_t timestamp;
    uint32_t pid;
    uint32_t tid;
    uint32_t event_type;
    uint64_t addr;
    uint64_t size;
    int64_t retval;
    uint32_t syscall_nr;
    char comm[MAX_COMM_LEN];
    char filename[MAX_FILENAME_LEN];
};

struct memory_alloc {
    uint64_t addr;
    uint64_t size;
    uint64_t timestamp;
    uint32_t pid;
};

#endif
```

- ثوابت:
 - **MAX_COMM_LEN**: حداکثر طول نام فرمان فرآیند را تعریف می‌کند (۱۶ بایت، منطبق با **TASK_COMM_LEN** هسته)
 - **MAX_FILENAME_LEN**: حداکثر طول نام فایل را تعریف می‌کند (۲۵۶ بایت، محدودیت رایج در بسیاری از فایل سیستم‌ها)

- **enum event_type**:
 - **EVENT_PROCESS_START**: نشان می‌دهد فرآیند جدیدی شروع شده است.
 - **EVENT_PROCESS_END**: نشان می‌دهد فرآیند پایان یافته است.
 - **EVENT_SYSCALL_ENTER**: ورود به فرخوانی سیستم را نشان می‌دهد.
 - **EVENT_SYSCALL_EXIT**: خروج از فرخوانی سیستم را نشان می‌دهد.
 - **EVENT_MEMORY_ALLOC**: رویداد تخصیص حافظه را نشان می‌دهد.
 - **EVENT_MEMORY_FREE**: رویداد آزادسازی حافظه را نشان می‌دهد.
 - **EVENT_PAGE_FAULT**: خطای صفحه را نشان می‌دهد.
 - **EVENT_MMAP**: عملیات نگاشت حافظه (**mmap**) را نشان می‌دهد.
 - **EVENT_MUNMAP**: عملیات عدم نگاشت حافظه (**munmap**) را نشان می‌دهد.
- این شمارش روشی تمیز برای تمایز انواع مختلف رویدادهایی که می‌توانند توسط پروفایلر ضبط شوند فراهم می‌کند.

- **struct event_data**:
 - **timestamp**: برچسب زمانی با دقت نانو ثانیه برای زمان وقوع رویداد
 - **pid**: شناسه فرآیند، فرآیندی که رویداد را تولید کرده
 - **tid**: شناسه نخ، نخ که رویداد را تولید کرده
 - **event_type**: نوع رویداد (از **enum** قبلی)
 - **addr**: آدرس حافظه (برای رویدادهای مرتبط به حافظه)
 - **size**: اندازه یک بایت (برای رویدادهای مرتبط به حافظه)
 - **retval**: مقدار بازگشتی (برای رویدادهای فراخوانی سیستم)
 - **syscall_nr**: شماره فراخوانی سیستم (برای رویدادهای فراخوانی سیستم)
 - **comm**: نام فرمان فرآیند
 - **filename**: نام فایل (برای رویدادهای شروع فرآیند)
- این ساختار برای انتقال اطلاعات در مورد رویدادها از برنامه‌های **eBPF** به برنامه فضای کاربر استفاده می‌شود. شامل فیلدهایی برای همه انواع رویدادها است. هرچند همه فیلدها برای هر نوع رویداد استفاده نمی‌شود.

- **struct memory_alloc**:
 - **addr**: آدرس حافظه تخصیص
 - **size**: اندازه تخصیص به بایت
 - **timestamp**: زمان وقوع تخصیص
 - **pid**: شناسه فرآیندی که تخصیص انجام داده
- این ساختار برای ردیابی تخصیص‌های حافظه برای تشخیص نشت استفاده می‌شود. با ثبت تخصیص‌ها و بررسی اینکه آیا بعداً آزاد می‌شوند، پروفایلر می‌تواند نشت‌های بالقوه حافظه را شناسایی کند.
- فایل هدر عمدهٔ مینیمال است و فقط آنچه برای ارتباط بین اجزاء **eBPF** و فضای کاربر لازم است تعریف می‌کند. این به حفظ جداسازی واضح نگرانی‌ها و کاهش خطر عدم تطابق نسخه کمک می‌کند.

برنامه eBPF

فایل `process_profiler.bpf.c` حاوی برنامه‌های eBPF است که در محیط هسته اجرا می‌شوند

• اعلان `map`ها:

```
struct {
    __uint(type, BPF_MAP_TYPE_RINGBUF);
    __uint(max_entries, 256 * 1024);
} events SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, MAX_ENTRIES);
    __type(key, __u32);
    __type(value, __u8);
} target_pids SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, MAX_ENTRIES);
    __type(key, __u64);
    __type(value, struct memory_alloc);
} memory_map SEC(".maps");

struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, MAX_ENTRIES);
    __type(key, __u32);
    __type(value, __u64)
} syscall_enter_time SEC(".maps");
```

این اعلان‌ها نقشه‌های eBPF استفاده شده توسط برنامه را تعریف می‌کنند:

- **events**: بافر حلقوی برای انتقال داده‌های رویداد به فضای کاربر
- **target_pids**: نقشه هش ردیابی PID هایی که باید نظارت شوند
- **memory_map**: نقشه هش ردیابی تخصیص‌های حافظه
- **syscall_enter_time**: نقشه هش ردیابی زمان ورود فراخوانی‌های سیستم (برای محاسبه مدت زمان)

```
static __always_inline bool is_target_pid(__u32 pid) {
    return bpf_map_lookup_elem(&target_pids, &pid) != NULL;
}
```

این تابع کمکی بررسی می‌کند که آیا PID مشخصی در حال پروفایل‌گیری است یا جستجوی آن در نقشه `target_pids` توسط همه برنامه‌های eBPF برای فیلتر کردن رویدادها فقط برای فرآیندهای هدف استفاده می‌شود.

```
SEC("tp/sched/sched_process_exec")
int trace_process_exec(void *ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;

    if (!is_target_pid(pid))
        return 0;

    struct event_data *event = bpf_ringbuf_reserve(&events,
        sizeof(*event), 0);
    if (!event)
        return 0;

    event->timestamp = bpf_ktime_get_ns();
    event->pid = pid;
    event->tid = bpf_get_current_pid_tgid() & 0xffffffff;
    event->event_type = EVENT_PROCESS_START;
    bpf_get_current_comm(event->comm, sizeof(event->comm));

    __builtin_memcpy(event->filename, event->comm, MAX_COMM_LEN);

    bpf_ringbuf_submit(event, 0);
    return 0;
}
```

این تابع به نقطه ردیابی `sched_process_exec` متصل می‌شود که هنگام شروع اجرای فرآیند فعال می‌شود. این کار را انجام می‌دهد:

1. شناسه فرآیند فعلی را دریافت می‌کند
2. بررسی می‌کند که آیا این `pid` نظارت می‌شود
3. فضایی در بافر حلقه‌ای برای یک رویداد رزرو می‌کند
4. رویداد را با اطلاعات مربوط به شروع فرآیند پر می‌کند
5. رویداد را به بافر حلقه‌ای ارسال می‌کند

توابع مشابهی برای سایر رویدادهای چرخه حیات فرآیند وجود دارد.

```

SEC("uprobe")
int trace_malloc(struct pt_regs *ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;

    if (!is_target_pid(pid))
        return 0;

    __u64 size = PT_REGS_PARM1(ctx);

    struct event_data *event = bpf_ringbuf_reserve(&events,
sizeof(*event), 0);
    if (!event)
        return 0;

    event->timestamp = bpf_ktime_get_ns();
    event->pid = pid;
    event->tid = bpf_get_current_pid_tgid() & 0xffffffff;
    event->event_type = EVENT_MEMORY_ALLOC;
    event->size = size;
    bpf_get_current_comm(event->comm, sizeof(event->comm));

    bpf_ringbuf_submit(event, 0);
    return 0;
}

SEC("uretprobe")
int trace_malloc_ret(struct pt_regs *ctx) {
    __u32 pid = bpf_get_current_pid_tgid() >> 32;

    if (!is_target_pid(pid))
        return 0;

    __u64 addr = PT_REGS_RC(ctx);
    if (addr == 0)
        return 0;

    struct memory_alloc alloc = {
        .addr = addr,
        .timestamp = bpf_ktime_get_ns(),
        .pid = pid,
        .size = 0,
    };

    bpf_map_update_elem(&memory_map, &addr, &alloc, BPF_ANY);
    return 0;
}

```

این توابع به uprobes روی ماژول `malloc` متصل می‌شوند تا تخصیص‌های حافظه را ردیابی کنند:

- `trace_malloc`: هنگام ورود `malloc` فراخوانی می‌شود و اندازه درخواست شده را ثبت می‌کند

- `trace_malloc_ret`: هنگام بازگشت `malloc` فراخوانی می‌شود و آدرس تخصیص یافته را در `memory_map` ثبت می‌کند

توابع مشابهی برای ردیابی فراخوانی‌های `free` و سایر عملیات حافظه وجود دارد.

برنامه eBPF از چندین تابع کمکی BPF استفاده می‌کند:

- `bpf_get_current_pid_tgid()`: شناسه فرآیند و رشته فعلی را دریافت می‌کند.
- `bpf_ktime_get_ns()`: زمان فعلی را به نانوثانیه دریافت می‌کند.
- `bpf_ringbuf_reserve()`: فضایی در بافر حلقوی رزرو می‌کند.
- `bpf_ringbuf_submit()`: رویدادی را به بافر حلقوی ارسال می‌کند.
- `bpf_map_lookup_elem()`: عنصری را در نقشه جستجو می‌کند.
- `bpf_map_update_elem()`: عنصری را در نقشه به‌روزرسانی می‌کند.
- `bpf_map_delete_elem()`: عنصری را از نقشه حذف می‌کند.
- `bpf_get_current_comm()`: نام فرمان فرآیند فعلی را دریافت می‌کند.

این کمک‌کننده‌ها به برنامه eBPF اجازه تعامل با هسته و ارتباط با فضای کاربر را می‌دهند در حالی که امنیت و ایمنی حفظ می‌شود.

برنامه فضای کاربر

فایل `main.c` جز فضای کاربر پروفایلر فرآیند را پیاده‌سازی می‌کند. بیا ببینیم قسمت‌های کلیدی این فایل را بررسی کنیم:

```
static volatile bool exiting = false;
static struct process_profiler_bpf *skel;
static pid_t target_pid = 0;
static char target_program[256];
static char **target_args;

static struct {
    uint64_t syscalls;
    uint64_t memory_allocs;
    uint64_t memory_frees;
    uint64_t page_faults;
    uint64_t mmaps;
    uint64_t munmaps;
    uint64_t total_allocated;
    uint64_t total_freed;
```



```

uint64_t peak_memory;
uint64_t current_memory;

uint64_t minor_faults;
uint64_t major_faults;
uint64_t vsz;          // Virtual memory size
uint64_t rss;          // Resident set size
uint64_t rss_peak;
uint64_t swap_used;
} stats = {0};

```

این اعلان‌ها وضعیت سراسری برنامه را تعریف می‌کنند:

- **exiting**: پرچمی که نشان می‌دهد برنامه در حال خاموش شدن است
- **skel**: اشاره‌گری به اسکلت eBPF که برنامه‌های eBPF بارگذاری شده را مدیریت می‌کند
- **target_pid**: فرآیندی که پروفایل می‌شود
- **target_program** و **target_args**: برنامه‌ای که اگر در خط فرمان مشخص شده باشد اجرا می‌شود
- **stats**: ساختاری که آمار مختلفی از فرآیند پروفایل شده ردیابی می‌کند

```

struct syscall_map {
    int num;
    const char *name;
};

// ... [جداول فراخوانی سیستم برای اختصار حذف شده] ...

static const char *get_syscall_name(int syscall_nr) {
    #if IS_X86_ARCH
        for (int i = 0; x86_syscalls[i].name != NULL; i++) {
            if (x86_syscalls[i].num == syscall_nr) {
                return x86_syscalls[i].name;
            }
        }
    #endif

    #if IS_AARCH64_ARCH
        for (int i = 0; aarch64_syscalls[i].name != NULL; i++) {
            if (aarch64_syscalls[i].num == syscall_nr) {
                return aarch64_syscalls[i].name;
            }
        }
    #endif

    return "unknown_syscall";
}

```

این کد نگاشت شماره فراخوانی‌های سیستم به نام‌های قابل خواندن توسط انسان را مدیریت می‌کند و از شماره فراخوانی `get_syscall_name` پشتیبانی می‌کند. تابع `AArch64` و `x86_64` معماری‌های سیستم را در جدول مخصوص معماری مناسب جستجو می‌کند و نام متناظر را برمی‌گرداند.

```
static int handle_event(void *ctx __attribute__((unused)), void *data,
size_t data_sz __attribute__((unused))) {
    const struct event_data *e = data;

    printf("[%11u.%0911u] PID: %u TID: %u COMM: %s EVENT: ",
           e->timestamp / 1000000000ULL, e->timestamp % 1000000000ULL,
           e->pid, e->tid, e->comm);

    switch (e->event_type) {
        case 0: // EVENT_PROCESS_START
            printf("PROCESS_START FILENAME: %s\n", e->filename);
            break;
        case 1: // EVENT_PROCESS_END
            printf("PROCESS_END\n");
            break;
        case 2: // EVENT_SYSCALL_ENTER
            {
                const char *syscall_name = get_syscall_name(e-
>syscall_nr);
                char args_buffer[256];
                format_syscall_args(syscall_name, e->args, args_buffer,
sizeof(args_buffer));
                printf("SYSCALL_ENTER SYSCALL: %s (%u) %s\n",
                       syscall_name, e->syscall_nr, args_buffer);
                stats.syscalls++;
            }
            break;
        // ... [سایر انواع رویداد برای اختصار حذف شده] ...
    }

    return 0;
}
```

این تابع هر زمان که رویدادی از بافر حلقوی `eBPF` دریافت می‌شود فراخوانی می‌شود. این کار را انجام می‌دهد

- هدری با زمان، `PID`، `TID` و نام فرآیند چاپ می‌کند
- رویداد را بر اساس نوع آن مدیریت می‌کند و اطلاعات مربوطه را چاپ می‌کند
- آمار را در صورت لزوم به‌روزرسانی می‌کند

```

static int attach_uprobe_malloc_free(struct process_profiler_bpf *skel,
pid_t pid __attribute__((unused))) {
    const char *libc_path = "/lib/aarch64-linux-gnu/libc.so.6";

    // ... [برای اختصار حذف شده libc تشخیص مسیر] ...

    unsigned long malloc_offset = 0;
    FILE *fp;
    char cmd[512];
    char line[256];

    snprintf(cmd, sizeof(cmd), "objdump -T %s | grep ' malloc$' | awk
'{print $1}'", libc_path);
    fp = popen(cmd, "r");
    if (fp != NULL) {
        if (fgets(line, sizeof(line), fp) != NULL) {
            malloc_offset = strtoul(line, NULL, 16);
            printf("Found malloc at offset: 0x%lx\n", malloc_offset);
        }
        pclose(fp);
    }

    // ... [برای اختصار حذف شده free_offset کد مشابه برای] ...

    // صحیح libbpf API با استفاده از uprobe malloc اتصال
    if (malloc_offset > 0) {
        // برای اتصال به همه فرآیندها PID استفاده از 1- برای
        skel->links.trace_malloc = bpf_program__attach_uprobe(skel-
>progs.trace_malloc,
                                                                    false, -1,
libc_path,
                                                                    malloc_offset);

        if (!skel->links.trace_malloc) {
            fprintf(stderr, "Warning: Failed to attach malloc uprobe\n");
        } else {
            printf("Successfully attached malloc uprobe\n");

            // موفق بود اگر uprobe malloc uretprobe اتصال
            skel->links.trace_malloc_ret =
bpf_program__attach_uprobe(skel->progs.trace_malloc_ret,
                                                                    true, -
1, libc_path,
                                                                    malloc_offset);

            if (!skel->links.trace_malloc_ret) {
                fprintf(stderr, "Warning: Failed to attach malloc
uretprobe\n");
            } else {
                printf("Successfully attached malloc uretprobe\n");
            }
        }
    }
}

```

```
... [برای اختصار حذف شده uprobe free کد مشابه برای] ...  
return 0;  
}
```

این تابع `uprobe` ها را به توابع `malloc` و `free` در کتابخانه C متصل می کند تا تخصیص ها و آزادسازی های حافظه را ردیابی کند. این کار را انجام می دهد:

- مسیر کتابخانه C را تعیین می کند
- از `objdump` برای یافتن `offset` های توابع `malloc` و `free` استفاده می کند
- `uprobe` ها و `uretprobe` ها را به این توابع با استفاده از `libbbpf` API متصل می کند

```

static pid_t spawn_target_program() {
    int sync_pipe[2];
    if (pipe(sync_pipe) == -1) {
        perror("pipe");
        return -1;
    }

    pid_t pid = fork();

    if (pid == 0) {
        close(sync_pipe[1]);

        char buf;
        if (read(sync_pipe[0], &buf, 1) != 1) {
            perror("Child read from pipe failed");
            exit(1);
        }
        close(sync_pipe[0]);

        execvp(target_program, target_args);
        perror("execvp");
        exit(1);
    } else if (pid > 0) {
        close(sync_pipe[0]);

        printf("Spawned target process with PID: %d\n", pid);

        printf("Attaching BPF programs before starting process...\n");
        sleep(1);

        if (write(sync_pipe[1], "x", 1) != 1) {
            perror("Parent write to pipe failed");
            return -1;
        }
        close(sync_pipe[1]);

        return pid;
    } else {
        perror("fork");
        return -1;
    }
}

```

این تابع در صورتی که برنامه‌ای در خط فرمان مشخص شده باشد، فرآیند جدیدی برای پروفایل‌گیری تولید می‌کند. این کار را انجام می‌دهد:

1. pipe برای همگام‌سازی بین فرآیندهای والد و فرزند ایجاد می‌کند

2. فرآیند جدیدی fork می‌کند

3. در فرآیند فرزند:

- منتظر سیگنال از والد قبل از شروع اجرا می‌ماند

- برنامه هدف را اجرا می‌کند

4. در فرآیند والد:

- منتظر اتصال برنامه‌های eBPF می‌ماند

- به فرزند سیگنال شروع اجرا می‌دهد

- PID فرآیند فرزند را برمی‌گرداند

این رویکرد اطمینان می‌دهد که برنامه‌های eBPF قبل از شروع اجرای فرآیند هدف متصل شده‌اند و همه رویدادها را از ابتدا ضبط می‌کنند.

```
int main(int argc, char **argv) {
    struct ring_buffer *rb = NULL;
    int err;

    // ... [تجزیه آرگومان‌های خط فرمان برای اختصار حذف شده] ...

    signal(SIGINT, sig_handler);
    signal(SIGTERM, sig_handler);

    struct rlimit rlim_new = {
        .rlim_cur = RLIM_INFINITY,
        .rlim_max = RLIM_INFINITY,
    };
    if (setrlimit(RLIMIT_MEMLOCK, &rlim_new)) {
        fprintf(stderr, "Failed to increase RLIMIT_MEMLOCK limit!\n");
        return 1;
    }

    skel = process_profiler_bpf__open();
    if (!skel) {
        fprintf(stderr, "Failed to open BPF skeleton\n");
        return 1;
    }

    err = process_profiler_bpf__load(skel);
    if (err) {
        fprintf(stderr, "Failed to load and verify BPF skeleton\n");
        goto cleanup;
    }

    err = process_profiler_bpf__attach(skel);
    if (err) {
```

```

        fprintf(stderr, "Failed to attach BPF skeleton\n");
        goto cleanup;
    }

    // ... [و مدیریت فرآیند هدف برای اختصار حذف شده uprobes اتصال] ...

    rb = ring_buffer__new(bpf_map__fd(skel->maps.events), handle_event,
        NULL, NULL);
    if (!rb) {
        err = -1;
        fprintf(stderr, "Failed to create ring buffer\n");
        goto cleanup;
    }

    printf("Successfully started profiling PID %d\n", target_pid);

    while (!exiting) {
        err = ring_buffer__poll(rb, 100);
        if (err == -EINTR) {
            err = 0;
            break;
        }
        if (err < 0) {
            printf("Error polling ring buffer: %d\n", err);
            break;
        }

        if (kill(target_pid, 0) == -1 && errno == ESRCH) {
            printf("Target process has exited\n");
            break;
        }

        static int counter = 0;
        if (++counter % 10 == 0 && target_pid > 0 && kill(target_pid, 0)
== 0) {
            collect_proc_stats(target_pid);
            counter = 0;
        }
    }

    // ... [پاکسازی و گزارش‌دهی برای اختصار حذف شده] ...

cleanup:
    ring_buffer__free(rb);
    process_profiler_bpf__destroy(skel);
    return err < 0 ? -err : 0;
}

```

تابع اصلی کل فرآیند پروفایل‌گیری را هماهنگ می‌کند:

1. آرگومان‌های خط فرمان را تجزیه می‌کند تا فرآیند هدف را تعیین کند
 2. کنترل‌کننده‌های سیگنال برای پایان مناسب تنظیم می‌کند
 3. محدودیت قفل حافظه را افزایش می‌دهد تا ایجاد نقشه‌های eBPF امکان‌پذیر شود
 4. برنامه eBPF را باز می‌کند، بارگذاری می‌کند و متصل می‌کند
 5. uprobes را برای ردیابی تخصیص‌های حافظه متصل می‌کند
 6. در صورت مشخص بودن، فرآیند هدف را تولید می‌کند
 7. بافر حلقوی برای دریافت رویدادها از برنامه eBPF ایجاد می‌کند
 8. وارد حلقه رویداد اصلی می‌شود، رویدادها را نظرسنجی می‌کند و پردازش می‌کند
 9. به طور دوره‌ای آمار اضافی از *proc* / جمع‌آوری می‌کند
 10. هنگام پایان، منابع را پاکسازی می‌کند و گزارش نهایی تولید می‌کند
- تابع اصلی قلب برنامه فضای کاربر است که همه اجزاء دیگر را به هم پیوند می‌دهد و فرآیند کلی پروفایل‌گیری را مدیریت می‌کند.

ساخت برنامه

وابستگی‌های مورد نیاز را نصب کنید:

```
make install-deps
```

پروفایلر را بسازید:

```
make
```

برنامه را اجرا کنید:

```
./process_profiler {path to an executable}
```

یا یک برنامه در حال اجرا:

```
./process_profiler -p {pid}
```

سپس بعد دیدن اطلاعات بعد از مشاهدات، کنترل سی را بزنید

درک خروجی

جریان ورود

```
[2170.911528539] PID: 5575 TID: 5575 COMM: a EVENT: SYSCALL_ENTER  
SYSCALL: write (64) args=[fd=1, buf=0xc254991f82a0, count=6]  
[2170.911576931] PID: 5575 TID: 5575 COMM: a EVENT: SYSCALL_EXIT RETVAL:  
6
```

این رویدادهای جداگانه را همان‌طور که رخ می‌دهند با زمان‌ها، اطلاعات فرآیند و جزئیات خاص رویداد نشان می‌دهد.

آمار پروفایل‌گیری

```
=== PROFILING STATISTICS ===  
System calls: 32  
Memory allocations: 5  
Memory frees: 0  
Page faults: 91  
Memory mappings (mmap): 0  
Memory unmappings (munmap): 0  
Total allocated: 5120 bytes  
Total freed: 0 bytes  
Peak memory usage: 5120 bytes  
Current memory usage: 5120 bytes  
Potential memory leaks: 5 allocations not freed
```

نشت‌های حافظه

```
=== MEMORY LEAK ANALYSIS ===  
Potential memory leaks detected:  
LEAK: addr=0xc3149caf3ed0 size=0 pid=89286 timestamp=5048494450657  
LEAK: addr=0xc3149caf36b0 size=0 pid=89286 timestamp=5048494441247  
LEAK: addr=0xc3149caf3ac0 size=0 pid=89286 timestamp=5048494444869  
LEAK: addr=0xc3149caf32a0 size=0 pid=89286 timestamp=5048494433086  
LEAK: addr=0xc3149caf42e0 size=0 pid=89286 timestamp=5048494454321
```

این بخش تخصیص‌های حافظه‌ای را که آزاد نشده‌اند شناسایی می‌کند که می‌تواند نشان‌دهنده نشت حافظه باشد.

آمار صفحه‌بندی حافظه

```
=== MEMORY PAGING STATISTICS ===  
Minor page faults: 70  
Major page faults: 0  
Virtual memory size: 0 bytes (0.00 MB)  
Resident set size: 5120 bytes (0.00 MB)  
Peak resident set size: 5120 bytes (0.00 MB)  
Page fault rate: 1.30 faults/sec  
Memory efficiency:  
- Allocation efficiency: 100.00%
```

```

ubuntu@ubuntu: ~/process_profiler
ubuntu@ubuntu:~/process_profiler$ make install-deps
sudo apt update
Hit:1 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Hit:2 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease
Hit:3 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
sudo apt install -y libbpf-dev clang llvm linux-tools-common linux-headers-6.8.0-60-generic
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libbpf-dev is already the newest version (1:1.3.0-2build2).
clang is already the newest version (1:18.0-59-exp2).
llvm is already the newest version (1:18.0-59-exp2).
linux-tools-common is already the newest version (6.8.0-60.63).
linux-headers-6.8.0-60-generic is already the newest version (6.8.0-60.63).
linux-headers-6.8.0-60-generic set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ubuntu:~/process_profiler$
  
```

```

ubuntu@ubuntu: ~/process_profiler
ubuntu@ubuntu:~/process_profiler$ make
bpftrace btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
clang -g -O2 -target bpf -D__TARGET_ARCH_arm64 -I. -c process_profiler.bpf.c -o process_profiler.bpf.o
bpftrace gen skeleton process_profiler.bpf.o > process_profiler.skel.h
clang -O2 -Wall -Wextra -I/usr/include/bpf -I./include main.c -lbpf -lelf -lz -o process_profiler
ubuntu@ubuntu:~/process_profiler$
  
```

```
ubuntu@ubuntu: ~/process_profiler
ubuntu@ubuntu:~/process_profiler$ sudo ./process_profiler /tmp/a
Found malloc at offset: 0x965b0
Found free at offset: 0x96eb0
Successfully attached malloc uprobe
Successfully attached malloc uretprobe
Successfully attached free uprobe
Successfully attached free uretprobe
Spawned target process with PID: 93338
Attaching BPF programs before starting process...
Added PID 93338 to target map
Successfully started profiling PID 93338
[6681.658789971] PID: 93338 TID: 93338 COMM: process_profiler EVENT: SYSCALL_EXIT
RETVAL: 1
[6681.658843720] PID: 93338 TID: 93338 COMM: process_profiler EVENT: SYSCALL_ENTER
R SYSCALL: close (57) args=[fd=47]
[6681.658869720] PID: 93338 TID: 93338 COMM: process_profiler EVENT: PAGE_FAULT A
DDR: 0x0
[6681.658876387] PID: 93338 TID: 93338 COMM: process_profiler EVENT: PAGE_FAULT A
DDR: 0x0
[6681.658911220] PID: 93338 TID: 93338 COMM: process_profiler EVENT: SYSCALL_EXIT
RETVAL: 0
[6681.658939844] PID: 93338 TID: 93338 COMM: process_profiler EVENT: SYSCALL_ENTER
R SYSCALL: execve (221) args=[0xc6090c9301c0, 0xffffffff37b2cb0, 0xffffffff37b2cc0, 0x
31824324c0720, 0xfe3874078b50, 0x1]
[6681.659000052] PID: 93338 TID: 93338 COMM: process_profiler EVENT: PAGE_FAULT A
```

```
Target process exited with status: 0
=== PROFILING STATISTICS ===
System calls: 32
Memory allocations: 5
Memory frees: 0
Page faults: 91
Memory mappings (mmap): 0
Memory unmappings (munmap): 0
Total allocated: 5120 bytes
Total freed: 0 bytes
Peak memory usage: 5120 bytes
Current memory usage: 5120 bytes
Potential memory leaks: 5 allocations not freed

=== MEMORY LEAK ANALYSIS ===
Potential memory leaks detected:
LEAK: addr=0xbf31b1be42a0 size=0 pid=93338 timestamp=6681662268389
LEAK: addr=0xbf31b1be52e0 size=0 pid=93338 timestamp=6681662301847
LEAK: addr=0xbf31b1be4ac0 size=0 pid=93338 timestamp=6681662286138
LEAK: addr=0xbf31b1be46b0 size=0 pid=93338 timestamp=6681662280722
LEAK: addr=0xbf31b1be4ed0 size=0 pid=93338 timestamp=6681662293597

=== MEMORY PAGING STATISTICS ===
```